

EXPERIMENT – 1

Name: K ANSHU

Roll No: 2520030440

Section: 8

Title

Implementation of Process Creation and Command Execution in Linux using fork(), exec(), and wait() System Calls.

Aim

To develop a C program that accepts a Linux command from the user, creates a child process using fork(), executes the command, waits for the child process, and displays Parent and Child PIDs.

Theory / Algorithm / Code

Same as the reference document provided by the faculty/friend. Only student details and execution screenshots have been updated.

CODE

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

#include <string.h>

int main()

{

    char command[100];

    printf("Enter a Linux command: ");

    fgets(command, sizeof(command), stdin);

    // Remove newline character

    command[strcspn(command, "\n")] = '\0';
```

```
pid_t pid = fork();

if(pid < 0)
{
    printf("Fork Failed!\n");
    return 1;
}

else if(pid == 0)
{
    printf("\nChild Process\n");
    printf("Child PID : %d\n", getpid());
    printf("Parent PID: %d\n", getppid());

    execlp(command, command, NULL);

    printf("Command execution failed!\n");
    exit(1);
}

else
{
    printf("\nParent Process\n");
    printf("Parent PID: %d\n", getpid());
```

```

wait(NULL);

printf("Child process completed.\n");

}

return 0;
}

```

Execution Screenshots

Installation and successful compilation

Program execution using lscpu

Program execution using top

```

Processing triggers for systemd (259.5-0ubuntu3) ...
Processing triggers for man-db (2.13.1-1build1) ...
Processing triggers for libc-bin (2.43-2ubuntu2.3) ...
K_anshu@DESKTOP-TH208AQ:~$ ls /usr/include/stdio.h
/usr/include/stdio.h
K_anshu@DESKTOP-TH208AQ:~$ gcc us_command.c -o process
K_anshu@DESKTOP-TH208AQ:~$ ./process
Enter a Linux command: lscpu

Parent Process:
Parent PID: 1884

Child Process
Child PID : 1889
Parent PID: 1884
Architecture:          32-bit, 64-bit
CPU op-mode(s):        39 bits physical, 48 bits virtual
Address sizes:          16
Byte Order:             Little Endian
CPU(s):                 10
On-line CPU(s) list:   0-15
Vendor ID:              GenuineIntel
Model name:             13th Gen Intel(R) Core(TM) i5-13450HX
CPU family:             6
Model:                 183
Thread(s) per core:    2
Core(s) per socket:    8
Socket(s):              1
Stepping:               1
BogoMIPS:               5222.39
Flags:                  fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush vmx fxsr sse sse2 ss ht syscall nx pdptab rdtsc
cp ln constant_tsc rep_good nopl xtopology tsc_reliable nonstop_tsc edabletsoc known_freq pni pclmulqdq vmx ssse3fma cx16pcid
sse4_1 sse4_2 x2apic movbepopcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm3dnowprefetch ssbd ibrs i
bpb stibp tpr_shadowvffileptority ept vpid ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcidrdpid md_clearflush_lid
opt clwb sha_ni xsave ept xsavec xgetbv1 xsavec avx_vnni vnni unip waitpkg gfni vaes vpclmulqdq rdpid movdir movdir64b fsrm rq
clear serialize ibrt flush_lid arch_capabilities

Virtualization features:
Virtualization:         VT-x
Hypervisor vendor:      Microsoft
Virtualization type:    full

```

Result

The program was executed successfully. Linux commands were executed through the child process and the parent process waited until completion.